

Lab 5

Part A – Manual Exercises

Task 1: Translation

Question 1: A quadrilateral has vertices at the points: P(0, 0), Q(6, 0), R(6, 4), and S(0, 4). Apply a translation with the translation vector $T_x = 2$ and $T_y = -5$. What are the new coordinates of the quadrilateral after translation?

- P'(2, -5)
- Q'(8, -5)
- R'(8, -1)
- S'(2, -1)

Question 2: On a map grid, a treasure chest is located at point T(150, 80). The instructions state: "Move 50 units East and 30 units North to find the key." Assuming East is the positive x-direction and North is the positive y-direction, what are the coordinates of the key?

- Key's coordinates = $(150 + 50, 80 + 30) = (200, 110)$

Question 3: After a translation, a triangle's vertices are at A'(0, -2), B'(3, 5), and C'(-1, 1). The translation vector used was $T_x = -2$, $T_y = 4$. What were the original coordinates of the triangle before the translation?

- Original = Final – Vector
 $(x, y) = (x' - (-2), y' - 4) = (x' + 2, y' - 4)$
- A: $(0 + 2, -2 - 4) = (2, -6)$
- B: $(3 + 2, 5 - 4) = (5, 1)$
- C: $(-1 + 2, 1 - 4) = (1, -3)$
- Original triangle: A(2, -6), B(5, 1), C(1, -3)

Task 2: Scaling

Question 4: A quadrilateral has vertices at P(2, 3), Q(5, 3), R(5, 6), and S(2, 6). Apply scaling with $S_x = 3$ and $S_y = 2$ about the fixed point (4, 5). Find the new coordinates of the quadrilateral.

- Scale $S_x = 3$, $S_y = 2$ about F(4,5)
 - P'(-2, 1)
 - Q'(7, 1)
-

- $R'(7, 7)$
- $S'(-2, 7)$

Question 5: A triangle has vertices at $A(4, 2)$, $B(1, -1)$, and $C(3, -3)$. 5 Apply scaling with $S_x = 0.5$ and $S_y = 1.5$ about the fixed point $(0, 0)$. Find the new coordinates of the triangle.

- Scale $S_x = 0.5$, $S_y = 1.5$ about $F(0,0)$
- $A'(-3, -2)$
- $B'(-5, -4)$
- $C'(-6, -1)$

Task 3: Rotation

Question 6: A triangle has vertices at $A(3, 2)$, $B(5, 4)$, and $C(6, 1)$. Rotate the triangle by 180 degrees counterclockwise about the origin $(0, 0)$. What are the new coordinates of the triangle after rotation?

$(180^\circ \text{ about origin} \Rightarrow (x', y') = (-x, -y))$

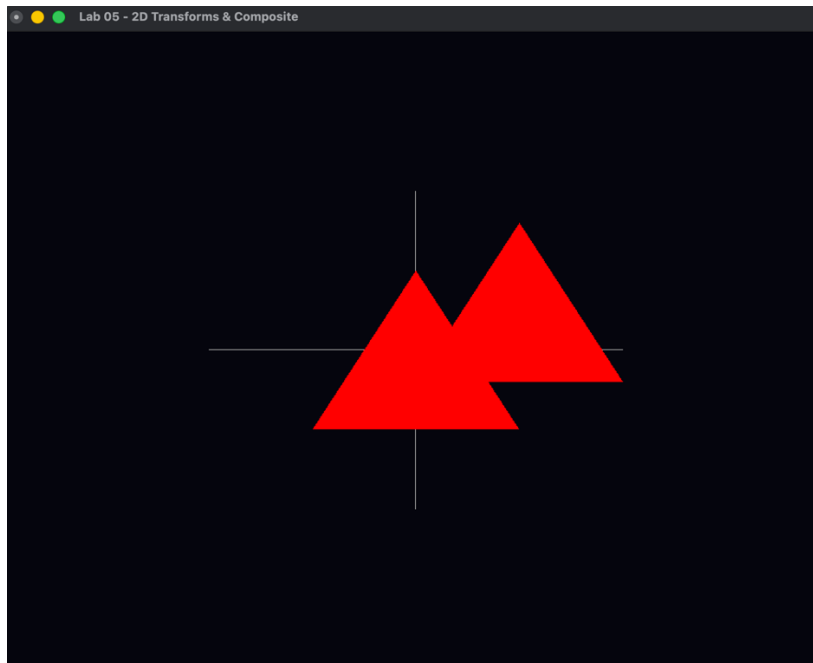
- $A'(-3, -2)$
- $B'(-5, -4)$
- $C'(-6, -1)$

Question 7: A triangle has vertices at $A(2, 3)$, $B(5, 3)$, and $C(5, 6)$. Rotate the triangle by 90 degrees counterclockwise about the fixed point $(3, 3)$. What are the new coordinates?

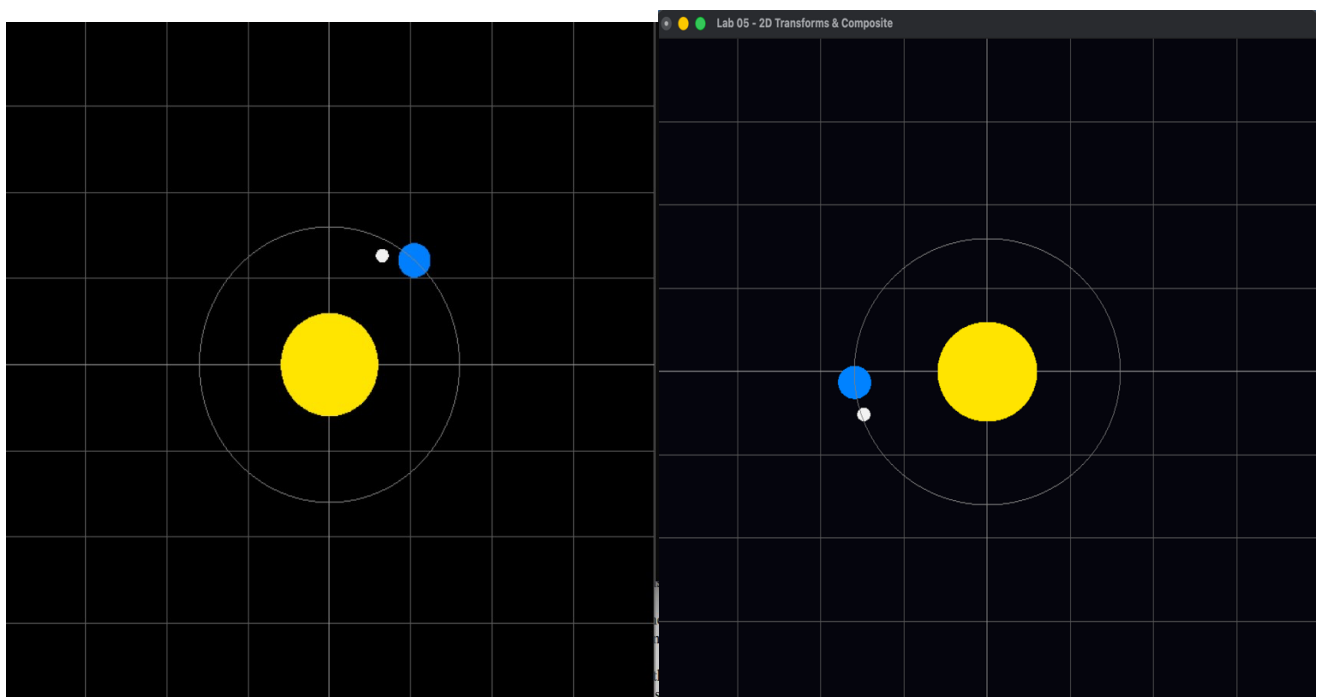
- Rotate 90° CCW about $F(3, 3)$
 - $A'(3, 2)$
 - $B'(3, 5)$
 - $C'(0, 5)$
-

Part B – Coding Exercise

Task 4: Implement Basic Transformations



Task 5: Composite Transformation



Code -

```

#include <GLUT/glut.h>
#include <math.h>
#include <stdio.h>

// Simple red triangle
void drawTriangle(void) {
    glColor3f(1.0f, 0.0f, 0.0f);           // red
    glBegin(GL_TRIANGLES);
        glVertex2f(-0.5f, -0.5f);         // bottom-left
        glVertex2f( 0.5f, -0.5f);         // bottom-right
        glVertex2f( 0.0f,  0.5f);         // top
    glEnd();
}

void drawAxes() {
    glLineWidth(1.0f);
    glColor3f(0.6f, 0.6f, 0.6f);
    glBegin(GL_LINES);
        glVertex2f(-1.0f, 0.0f); glVertex2f(1.0f, 0.0f); // X
        glVertex2f(0.0f, -1.0f); glVertex2f(0.0f, 1.0f); // Y
    glEnd();
}

// 1) Translation demo
void Translate() {
    glClear(GL_COLOR_BUFFER_BIT);
    drawAxes();

    // original
    drawTriangle();

    // translated
    glPushMatrix();
        glTranslatef(0.5f, 0.3f, 0.0f);
        glColor3f(0.0f, 0.7f, 1.0f);     // cyan for transformed
        drawTriangle();
    glPopMatrix();

    glutSwapBuffers();
}

// 2) Scaling about origin
void ScaleAboutOrigin() {
    glClear(GL_COLOR_BUFFER_BIT);
    drawAxes();

    // original
    drawTriangle();

    // scaled (about origin)

```

```

    glPushMatrix();
        glScalef(2.0f, 1.5f, 1.0f);
        glColor3f(0.0f, 0.7f, 1.0f);
        drawTriangle();
    glPopMatrix();

    glFlush();
}

// 3) Scaling about a fixed point (0.25, 0.25)
void ScaleAboutFixedPoint() {
    glClear(GL_COLOR_BUFFER_BIT);
    drawAxes();

    // original
    drawTriangle();

    // move fixed point to origin -> scale -> move back
    glPushMatrix();
        glTranslatef(0.25f, 0.25f, 0.0f);
        glScalef(1.8f, 1.4f, 1.0f);
        glTranslatef(-0.25f, -0.25f, 0.0f);
        glColor3f(0.0f, 0.7f, 1.0f);
        drawTriangle();
    glPopMatrix();

    glFlush();
}

// 4) Rotation about origin (45°)
void RotationAboutOrigin() {
    glClear(GL_COLOR_BUFFER_BIT);
    drawAxes();

    // original
    drawTriangle();

    // rotated about origin
    glPushMatrix();
        glRotatef(45.0f, 0.0f, 0.0f, 1.0f);
        glColor3f(0.0f, 0.7f, 1.0f);
        drawTriangle();
    glPopMatrix();

    glFlush();
}

// 5) Rotation about a fixed point (0,0) (for demo we rotate the triangle
after shifting it)
void RotationAboutFixedPoint() {
    glClear(GL_COLOR_BUFFER_BIT);
    drawAxes();

    // put triangle somewhere so (0,0) lies inside its base line

```

```

    glPushMatrix();
        glTranslatef(0.3f, 0.0f, 0.0f);
        drawTriangle();
    glPopMatrix();

    // rotate about fixed point (0,0): translate(-fp) -> rotate ->
    translate(+fp)
    glPushMatrix();
        glTranslatef(0.0f, 0.0f, 0.0f); // fp = (0,0)
        glRotatef(45.0f, 0.0f, 0.0f, 1.0f);
        glTranslatef(0.3f, 0.0f, 0.0f); // original shift of the triangle
        glColor3f(0.0f, 0.7f, 1.0f);
        drawTriangle();
    glPopMatrix();

    glFlush();
}

// Composite Transformation: Solar System
float earthOrbitAngle = 0.0f;
float earthRotationAngle= 0.0f;
float moonOrbitAngle = 0.0f;

void drawCircle(float cx, float cy, float r, int n, float R, float G, float
B) {
    glColor3f(R, G, B);
    glBegin(GL_TRIANGLE_FAN);
        glVertex2f(cx, cy);
        for (int i = 0; i <= n; ++i) {
            float theta = (2.0f * 3.14159265f * i) / n;
            float x = r * cosf(theta) + cx;
            float y = r * sinf(theta) + cy;
            glVertex2f(x, y);
        }
    glEnd();
}

void drawGrid() {
    glLineWidth(1.0f);
    glColor3f(0.35f,0.35f,0.35f);
    glBegin(GL_LINES);
        for (float x=-2.0f; x<=2.001f; x+=0.5f) { glVertex2f(x,-2.0f);
glVertex2f(x,2.0f); }
        for (float y=-2.0f; y<=2.001f; y+=0.5f) { glVertex2f(-2.0f,y);
glVertex2f(2.0f,y); }
    glEnd();

    // axes
    glColor3f(0.6f,0.6f,0.6f);
    glBegin(GL_LINES);
        glVertex2f(-2.0f,0.0f); glVertex2f(2.0f,0.0f);
        glVertex2f(0.0f,-2.0f); glVertex2f(0.0f,2.0f);

```

```

    glEnd();
}

void displaySolar() {
    glClear(GL_COLOR_BUFFER_BIT);

    drawGrid();

    // SUN (scaled at origin)
    glPushMatrix();
    glScalef(1.5f, 1.5f, 1.0f);
    drawCircle(0.0f, 0.0f, 0.2f, 48, 1.0f, 0.9f, 0.0f);
    glPopMatrix();

    // EARTH (orbit + translate + self-rotate)
    glPushMatrix();
    glRotatef(earthOrbitAngle, 0.0f, 0.0f, 1.0f); // orbit sun
    glTranslatef(0.8f, 0.0f, 0.0f); // orbital radius
    glPushMatrix();
    glRotatef(earthRotationAngle, 0.0f, 0.0f, 1.0f);
    drawCircle(0.0f, 0.0f, 0.1f, 36, 0.1f, 0.5f, 1.0f); // earth
    glPopMatrix();

    // MOON (relative to earth)
    glPushMatrix();
    glRotatef(moonOrbitAngle, 0.0f, 0.0f, 1.0f);
    glTranslatef(0.2f, 0.0f, 0.0f);
    drawCircle(0.0f, 0.0f, 0.04f, 24, 0.95f, 0.95f, 0.95f);
    glPopMatrix();
    glPopMatrix();

    // Earth orbit path
    glPushMatrix();
    glColor3f(0.5f, 0.5f, 0.5f);
    glBegin(GL_LINE_LOOP);
    for (int i=0; i<100; i++){
        float t = 2.0f*3.14159265f*i/100.0f;
        glVertex2f(0.8f*cosf(t), 0.8f*sinf(t));
    }
    glEnd();
    glPopMatrix();

    glutSwapBuffers();
}

void timer(int v) {
    earthOrbitAngle += 0.2f; // slow orbit
    earthRotationAngle += 1.0f; // faster spin
    moonOrbitAngle += 0.5f; // fast moon orbit
    glutPostRedisplay();
    glutTimerFunc(16, timer, 0); // ~60 FPS
}

```

```

void init() {

```

```

    glClearColor(0.02f,0.02f,0.05f,1.0f);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(-2.0, 2.0, -2.0, 2.0);
    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
}

void printInstructions() {
    printf("Lab 05: 2D Transforms & Composite Demo \n");
    printf("Use different display funcs to capture screenshots:\n");
    printf("  Translate / ScaleAboutOrigin / ScaleAboutFixedPoint\n");
    printf("  RotationAboutOrigin / RotationAboutFixedPoint\n");
    printf("Solar animation is shown by default.\n");
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGBA);
    glutInitWindowSize(900, 700);
    glutCreateWindow("Lab 05 - 2D Transforms & Composite");

    init();
    printInstructions();

    // Default: composite animation (solar system)
    glutDisplayFunc(RotationAboutFixedPoint);
    glutTimerFunc(0, timer, 0);

    glutMainLoop();
    return 0;
}

```